Check for updates

Data paper

The International Journal of Robotics Research
2021, Vol. 40(2-3) 515–520
© The Author(s) 2020
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/0278364920966642
journals.sagepub.com/home/ijr
SAGE

ALFA: A dataset for UAV fault and anomaly detection

Azarakhsh Keipour, Mohammadreza Mousaei and Sebastian Scherer

Abstract

We present a dataset of several fault types in control surfaces of a fixed-wing unmanned aerial vehicle (UAV) for use in fault detection and isolation (FDI) and anomaly detection (AD) research. Currently, the dataset includes processed data for 47 autonomous flights with 23 sudden full engine failure scenarios and 24 scenarios for 7 other types of sudden control surface (actuator) faults, with a total of 66 minutes of flight under normal conditions and 13 minutes of post-fault flight time. It additionally includes many hours of raw data of fully autonomous, autopilot-assisted and manual flights with tens of fault scenarios. The ground truth of the time and type of faults is provided in each scenario to enable evaluation of the methods using the dataset. We have also provided the helper tools in several programming languages to load and work with the data and to help the evaluation of a detection method using the dataset. A set of metrics is proposed to help to compare different methods using the dataset. Most of the current fault detection methods are evaluated in simulation and, as far as we know, this dataset is the only one providing the real flight data with faults in such capacity. We hope it will help advance the state of the art in AD or FDI research for autonomous aerial vehicles and mobile robots to enhance the safety of autonomous and remote flight operations further. The dataset and the provided tools can be accessed from https://doi.org/10.1184/R1/12707963.

Keywords

Dataset, fault detection and isolation, anomaly detection, unmanned aerial vehicles, autonomous robots, fixed-wing robots, engine failure, actuator failure, flight safety, evaluation metrics

1. Introduction

The recent growth in the use of autonomous aerial vehicles (AAVs) has increased concerns about the safety of the autonomous vehicles, the people, and the properties around the flight path and onboard the vehicle. To address the concerns, much research is being performed on new regulations, more robust systems are designed, and new systems and algorithms are introduced to detect the potential hardware and software issues.

Many methods have been introduced to detect hardware issues. These methods can be categorized in several ways: they can be learning-based or not, online or offline, identifying the fault type or detecting the anomaly. Each category has its pros and cons. For example, learning-based methods learn models for different fault types and can predict the faults with high precision. However, they have difficulty detecting new issues and are generally dependent on the availability of a large amount of training data, which is not always the case. Khalastchi and Kalech (2018) provide a useful review and comparison of different fault detection methods in robotics.

Collecting flight data from real aircraft to test a new fault detection and isolation (FDI) or anomaly detection (AD) method is a difficult task; the hardware is expensive, the tests are time-consuming, and imposing some of the fault types can lead to the loss of control of the vehicle. As a result, most of the proposed methods are only tested in simulation (Abbaspour et al., 2017; Khalastchi et al., 2013; Melody et al., 2000). The results reported by these methods may be very different from the real data, making a comparison between these methods with the other methods tested on real flight tests difficult. Even many of the methods tested on the real flight data only report a minimal number of tests (Bu et al., 2017; Lin et al., 2010; Sun et al., 2017) and only a few proposed methods have completed a reasonable number of tests on the real flight data (Keipour et al.,

Robotics Institute, Carnegie Mellon University, USA

Corresponding author:
Azarakhsh Keipour, Air Lab, Robotics Institute, Carnegie Mellon University, Pittsburgh, PA 15213, USA.
Email: keipour@cmu.edu

2019; Venkataraman et al., 2019). Providing a large dataset to the FDI and AD community working on unmanned aerial vehicles (UAVs) will open the opportunity to test the proposed methods on real data and to compare the results with other methods.

In this article, we present the Air Lab Fault and Anomaly (ALFA) dataset, which currently includes processed data for 47 autonomous flights with scenarios for 8 different types of sudden control surface faults, including engine, rudder, aileron, and elevator faults, with 23 of the scenarios focusing on full engine failures. The processed data consists of a total of 66 minutes of normal flight and 13 minutes of post-fault flight time. The dataset also includes several hours of raw autonomous, autopilot-assisted, and manual flight data with tens of different fault scenarios. The processed data provides the ground truth for the exact time and type of fault in each scenario to help with the evaluation of the new methods. A small portion of this dataset has been used by Keipour et al. (2019) in the evaluation of a real-time AD method. The current article describes the dataset in details and opens the access to the complete set of the processed and raw sequences along with the telemetry and dataflash log data of all the flights. In addition, we provide a set of helper codes for working with the processed data and helping with the evaluation of the new methods in C++, Python, and MATLAB languages. The dataset and the tools can be accessed from https://doi.org/10.1184/R1/12707963 (Keipour et al. (2020)).

The provided dataset can be useful for many types of FDI and AD methods that do not depend on the prior knowledge of the precise dynamic model of the robot. While the precise dynamics model of the robot is not provided, depending on the requirements of a specific method, some form of model can be estimated from the data or from the general airplane dynamics combined with the hardware description of the robot used for the tests.

In the next section, the platform used for the collection of the dataset and the changes needed to enable the creation of faults are described; Section 3 explains the details about the data and the usage of the dataset; Section 4 proposes the metrics for evaluation of new methods; finally, Section 5 summarizes the article and proposes ideas for the future work.

2. The platform

2.1. Hardware setup

The platform used for the collection of the dataset is a custom modification of the Carbon Z T-28 model plane. The plane has 2 m of wingspan, a single electric engine in the front, ailerons, flaperons, an elevator, and a rudder. We equipped the plane with a Holybro PX4 2.4.6 autopilot, a Pitot Tube, a GPS module, and an Nvidia Jetson TX2 onboard computer. In addition to the receiver, we also equipped it with a radio for communication with the ground station. Figure 1 shows the described platform.

2.2. Software

The Pixhawk autopilot uses a custom version of Ardupilot/ArduPlane firmware to control the plane in both manual and autonomous modes and to create the simulations. The original firmware is modified from ArduPlane v3.9.0beta1 to include four new parameters as follows.

DisableEngine: This parameter can disable the engine to simulate a complete engine failure.
DisableElevator: This parameter can fix the elevator in the horizontal position to simulate a stuck elevator.
DisableRudder: It can fix the rudder all the way to the left, right, or in the middle to simulate rudder hardover.
DisableAileron: It can fix the left aileron, right aileron, or both in the horizontal position to simulate stuck aileron(s).

For safety reasons, all the parameters are programmed to only work in the autonomous mode; at any time during the autonomous flight, the safety pilot can take over the control of the plane and all the disabled actuators and the engine will start working normally again. The commands for disabling the control surfaces (modifying the mentioned parameters in the autopilot) can only be sent through the ground control station (GCS). Figure 2 shows the communication between the pilot, UAV, and the GCS.

The onboard computer uses Robot Operating System (ROS) Kinetic Kame on Linux Ubuntu 16.04 (Xenial) to read the flight and state information from the Pixhawk using the MAVROS package (the MAVLink node for ROS). The data is recorded as a rosbag, and the ground truth about the faults is periodically published by a node that checks the status of the mentioned custom parameters. The autonomous flight uses a trajectory controller modified from the work of Schopferer et al. (2018) to enable the control using the onboard TX2 computer instead of the ground station.

Furthermore, to access information about the internal commands of the autopilot (e.g., commanded roll/pitch), both the firmware and MAVROS are modified to publish the desired information in high frequency using the MAVLink protocol.

3. Dataset

The presented dataset is entirely collected in an airport around Pittsburgh, Pennsylvania. Figure 3 shows the location of the tests as well as a sample trajectory used in the recorded autonomous flights. Each flight sequence usually includes only a portion of the full trajectory, which can be extracted from the data.

This section describes the data in the dataset in more details, lists the types of faults that are in the dataset, and discusses the provided tools to work with the data and to evaluate an FDI or AD method using the data.

3.1. Data formats

The dataset consists of four types of data, described as follows.

Fig. 1. The Carbon-Z T-28 fixed-wing UAV platform equipped with an onboard computer and additional modules for our dataset collection. This is the same platform used by Keipour et al. (2019) in a previous work.

Fig. 2. The communication between the safety pilot, the UAV, and the GCS. The pilot only takes over when the safety is going to be compromised. The GCS is used for disabling the desired control surfaces in the autonomous mode.

Fig. 3. The location of the data recorded and the trajectory used in some of the flight tests.

- *Autonomous flight sequences with failures*: Flight sequences processed to only contain autonomous flight data and to include failure ground-truth data only when there is a fault. Each file contains a flight sequence with at most one fault. The data is provided in three formats: comma-separated values (`csv`), MATLAB `mat`, and the original ROS `bag`.
- *Raw flight sequences*: Flight data for flights in all the modes without any preprocessing and is only provided in the original ROS `bag` format. Some files may include multiple failure test scenarios, whereas the others may contain no autonomous flight at all. All the files from the first category are cut from these files.
- *Telemetry logs from TX2*: All the telemetry data is recorded by the onboard TX2 computer from the tests without any preprocessing. The files do not contain the fault ground-truth information and can be useful for unsupervised detection methods. More information about the format is available on the ArduPilot website.[1]
- *Dataflash logs from Pixhawk*: All the data recorded on the Pixhawk autopilot from the tests without any preprocessing. The files do not contain the fault ground-truth information and can be useful for unsupervised detection methods. More information about the format is available on the ArduPilot website.

The directory structure of the dataset is shown in Figure 4. The main focus of this dataset is the first data type (the processed flight sequences), and the next few subsections describe these files in more detail.

3.2. Fault types

The types of the faults currently provided by the dataset are listed in Table 1. As can be seen, a large portion of the dataset is on engine failure, which is provided to help with the machine-learning-based methods. However, we tried to provide various faults in order to encourage the methods that work on multiple faults. Note that the provided failures still allow the recovery of the robot by the safety pilot and many failure types with a potential of complete loss are not included in the dataset (e.g., the elevator getting stuck all the way down).

3.3. Data description

The processed file sequences in `mat` and `bag` formats include all the available topics, whereas each `csv` file includes one topic.

Each sequence includes information received using the modified MAVROS (as explained in Section 2.2), including the GPS information, local and global state, and wind estimation. Most of the topics are inherited from the original non-modified MAVROS module. These topics are usually available at 4 Hz or higher, and their description can be viewed from the MAVROS website.[2]

In addition to the original MAVROS topics, high-frequency data (between 20 and 25 Hz) is provided on the measured (by sensors) and the commanded (by autopilot) roll, pitch, velocity, airspeed, and yaw. The names of these topics start with `mavros/nav_info/`[3]; for example, for roll the topic name is `mavros/nav_info/roll`.

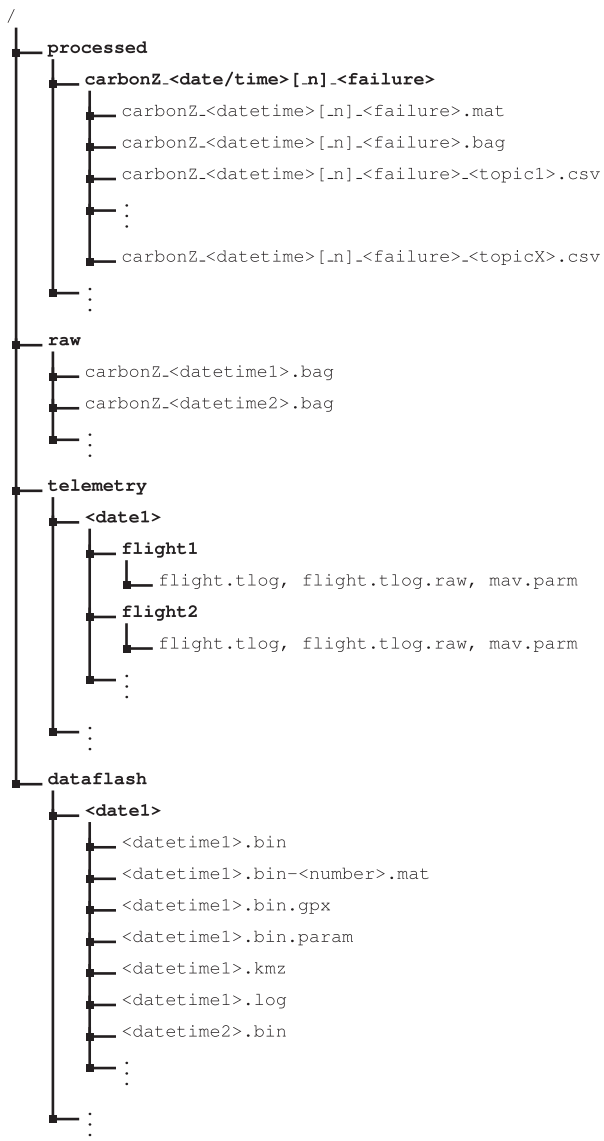

Fig. 4. The directory structure of the dataset.

At last, the ground-truth information is provided as topics for each control surface, starting with `failure_status/`. The topics are as follows.

- `failure_status/engines`: The value becomes `true` when engine failure happens.
- `failure_status/aileron`: The value becomes non-zero when an aileron failure happens. The value of *1* means the failure is on the right side; the value of *2* means that the left aileron is failed; the value of *3* means that both ailerons are failed. Failure here is defined as the aileron(s) getting stuck in zero position.
- `failure_status/rudder`: The value becomes non-zero when a rudder hardover happens. The value of *1* means that the rudder is stuck in zero position; the value of *2* means that it is stuck all the way to the left; the value of *3* means that the rudder is stuck all the way to the right.

Table 1. Fault types in the processed dataset.

Fault type	Number of test cases	Flight time before fault (s)	Flight time w/fault (s)
Engine full power loss	23	2,282	362
Rudder stuck to left	1	60	9
Rudder stuck to right	2	107	32
Elevator stuck at zero	2	181	23
Left aileron stuck at zero	3	228	183
Right aileron stuck at zero	4	442	231
Both ailerons stuck at zero	1	66	36
Rudder and aileron at zero	1	116	27
No fault	10	558	—
Total	47	3,935	777

- `failure_status/elevator`: The value becomes non-zero when an elevator failure happens. The value of *1* means that the elevator is stuck in zero position; the value of *2* means that it is stuck all the way down.

The ground-truth topics appear in a sequence only when there is a fault happening on that control surface. These topics are recorded at about 5 Hz rate; therefore, the first failure ground truth message can happen after up to 0.2 seconds after the exact moment of the fault.

Table 2 provides the list of the topics in the dataset sequences that provide potentially useful information for FDI and AD methods. Figure 5 shows a sample of the data for the moment when an engine failure happens. It also shows some of the topics in the data, including the additional data provided by the modified MAVROS.

3.4. Using the dataset

The `bag` files can easily be played back using the shell commands provided by the `rosbag` ROS package. The custom message definition files are provided to allow working with the topics defined by the custom message types in the `bag` files. In addition, the base tools are provided that allow working with the dataset using C++'11, MATLAB, and Python 3.x programming languages. The functionalities include loading a dataset file in memory, iterating through the whole dataset or a single topic in timestamp order, plotting a specific topic field, and some other methods such as separating the data for normal flight from the post-fault flight. There is no dependency on ROS or any other external package, and all the code is written using the standard libraries of these programming languages.

In addition, a base code is provided in C++'11 language to help with the evaluation of new fault and AD methods. It automatically subscribes to the ground-truth topics and waits for the method to publish the detection. It then returns information about the false positives, the delay in detection, and some other statistics.

Table 2. List of topics providing potentially useful information in the dataset sequences.

Field name	Description	Field type	Average rate
failure_status/engines	Engine failure status	std_msgs/Bool	5 Hz
failure_status/aileron	Aileron failure type	std_msgs/Int8	5 Hz
failure_status/rudder	Rudder failure type	std_msgs/Int8	5 Hz
failure_status/elevator	Elevator failure type	std_msgs/Int8	5 Hz
mavros/nav_info/airspeed	Airspeed information	mavros_msgs/NavDataPair[3]	20–25 Hz
mavros/nav_info/roll	Commanded and measured roll	mavros_msgs/NavDataPair[3]	20–25 Hz
mavros/nav_info/pitch	Commanded and measured pitch	mavros_msgs/NavDataPair[3]	20–25 Hz
mavros/nav_info/yaw	Commanded and measured yaw	mavros_msgs/NavDataPair[3]	20–25 Hz
mavros/nav_info/errors	Tracking, airspeed, and altitude errors	mavros_msgs/NavError[3]	20–25 Hz
mavros/nav_info/velocity	Commanded and measured velocity	mavros_msgs/NavVector[3]	20–25 Hz
mavctrl/path_dev	Path deviation (Schopferer et al., 2018)	geometry_msgs/Vector[3]	50 Hz
mavctrl/rpy	Measured roll, pitch, and yaw	geometry_msgs/Vector[3]	50 Hz
mavros/global_position/*	Global position info[2]	Various types	4–5 Hz
mavros/local_position/*	Local position info[2]	Various types	4 Hz
mavros/imu/*	IMU state[2]	Various types	10 Hz
mavros/setpoint_raw/*	Setpoint messages[2]	Various types	20–50 Hz
mavros/state	FCU state[2]	mavros_msgs/State	1 Hz
mavros/wind_estimation	Wind estimation by FCU[2]	geometry_msgs/TwistStamped	2–3 Hz
mavros/vfr_hud	Data for HUD[2]	mavros_msgs/VFR_HUD	2–3 Hz
mavros/time_reference	Time reference from SYSTEM_TIME[2]	sensor_msgs/TimeReference	2 Hz

Note: Superscript numbers correspond to the main article notes.
HUD, head-up display; IMU, inertial measurement unit.

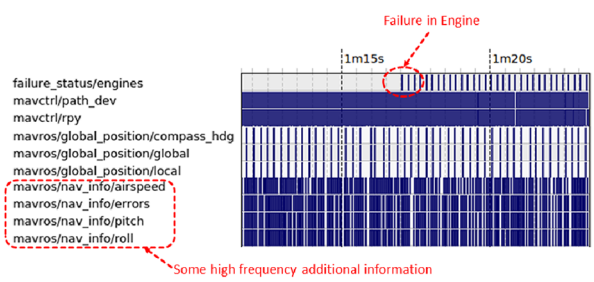

Fig. 5. A sample portion of a data file showing the moment that an engine failure happens and some additional high-frequency information provided.

4. Evaluation metrics

The metrics used for evaluation of a method can vary based on the method's class, the fault types, and the method applications. For example, for online methods, the delay between the fault happening and the detection can be critical for the safety of the flight, whereas for the offline methods this metric may not be as important and in many cases (e.g., for fault classification methods) it can be meaningless. We propose the following metrics for the evaluation of methods using the provided dataset.

- *Maximum detection time*: For online methods, the delay between the time a fault happens and the time it is detected is an important factor when comparing two methods. In real applications, a large detection delay in any scenario can lead to irreversible situations, resulting in the complete loss of control of the vehicle. Therefore, the maximum detection time is a useful metric for the evaluation of online methods.
- *Average detection time*: The average detection time over the set of fault scenarios shows the overall time performance of a method in detecting faults and is also a useful metric for the evaluation of the online methods.
- *Accuracy*: This metric is the ratio of the number of correctly classified sequences to the total number of sequences. Any false result (false fault detection or not detecting a fault) is considered as a misclassified case. This metric considers all the positive and negative sequences and is suitable to obtain an overall idea about the performance of an algorithm, but works best when the false detections and false negatives have similar costs.
- *Precision*: This metric is the ratio of the sequences with correctly predicted faults to the total number of detections (both true- and false-positive detections). Each sequence containing a false detection counts as a false positive and each sequence containing only correct detection(s) counts as a true positive. This metric indicates how reliable is the method when it announces a fault.
- *Recall*: This metric is the ratio of the sequences with correctly predicted faults to the total number of sequences containing fault(s). Each sequence containing only correct detection(s) counts as a true positive. This metric indicates how reliable is the method in detecting the faults.

5. Summary and future work

In this article, we have presented the ALFA dataset that provides autonomous flight data of a fixed-wing UAV with scenarios of different control surface faults in the middle of

the flights. We believe that this dataset will be highly useful in FDI and AD research.

The presented ALFA dataset is the most extensive dataset for fault detection and AD in AAVs, but it is by no means a complete dataset. The dataset contains several sudden control surface failures, but many more types of faults can happen in UAVs, including issues in sensors and gradual errors. We invite other groups and researchers to contribute to the dataset by providing their test data with other types of faults and platforms. It will significantly increase the speed of research in this area and will help with benchmarking future methods.

We have provided base codes to help with using the dataset and evaluation of the new methods with it. To further extend the usefulness of the data, it will be beneficial to create a benchmark to provide researchers with a better tool to compare their methods with the state-of-the-art.

Acknowledgements

This project became possible due to the support of Near Earth Autonomy (NEA). Also, the authors would like to thank Mark DeLouis for his help as the pilot during the months of testing and recording the data.

Funding

The authors disclosed receipt of the following financial support for the research, authorship and/or publication of this article: This work was supported through NASA Grant Number NNX17CL06C.

Notes

1. See http://ardupilot.org/plane
2. See http://wiki.ros.org/mavros
3. Custom message type provided with the support code (see Section 3.4).

ORCID iD

Azarakhsh Keipour https://orcid.org/0000-0002-7237-1885

References

Abbaspour A, Aboutalebi P, Yen KK and Sargolzaei A (2017) Neural adaptive observer-based sensor and actuator fault detection in nonlinear systems: Application in UAV. *ISA Transactions* 67: 317–329.

Bu J, Sun R, Bai H, et al. (2017) Integrated method for the UAV navigation sensor anomaly detection. *IET Radar, Sonar Navigation* 11(5): 847–853.

Keipour A, Mousaei M and Scherer S (2019) Automatic real-time anomaly detection for autonomous aerial vehicles. In: *2019 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5679–5685.

Keipour A, Mousaei M and Scherer S (2020) ALFA: A dataset for UAV fault and anomaly detection. DOI: 10.1184/R1/12707963.

Khalastchi E and Kalech M (2018) On fault detection and diagnosis in robotic systems. *ACM Computing Surveys* 51(1): 9:1–9:24.

Khalastchi E, Kalech M and Rokach L (2013) Sensor fault detection and diagnosis for autonomous systems. In: *Proceedings of the 2013 International Conference on Autonomous Agents and Multi-agent Systems (AAMAS '13)*. Richland, SC: International Foundation for Autonomous Agents and Multiagent Systems, pp. 15–22.

Lin R, Khalastchi E and Kaminka GA (2010) Detecting anomalies in unmanned vehicles using the Mahalanobis distance. In: *2010 IEEE International Conference on Robotics and Automation*, pp. 3038–3044.

Melody J, Basar T, Perkins W and Voulgaris P (2000) Parameter identification for inflight detection and characterization of aircraft icing. *Control Engineering Practice* 8(9): 985–1001.

Schopferer S, Lorenz JS, Keipour A and Scherer S (2018) Path planning for unmanned fixed-wing aircraft in uncertain wind conditions using trochoids. In: *2018 International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 503–512.

Sun R, Cheng Q, Wang G and Ochieng WY (2017) A novel online data-driven algorithm for detecting uav navigation sensor faults. *Sensors* 17(10): 2243.

Venkataraman R, Bauer P, Seiler P and Vanek B (2019) Comparison of fault detection and isolation methods for a small unmanned aircraft. *Control Engineering Practice* 84: 365–376.